

PID Control: From Analysis to Practice

ME 2801 — Week 5

Contents

1	PID Control: From Analysis to Practice	3
1.1	Introduction	3
1.2	The PID Controller	4
1.2.1	Intuition Before Math	4
1.2.2	Transfer Function Form	5
1.2.3	Closed-Loop Transfer Function	5
1.3	Proportional Control on a First-Order Plant	6
1.3.1	The USV Surge Model	6
1.3.2	Closed-Loop with P Only	6
1.4	Adding Integral Action	7
1.4.1	Why Integral Eliminates Steady-State Error	7
1.4.2	PI on the Surge Plant	8
1.4.3	Integral Windup	8
1.5	Derivative Action	9
1.5.1	Derivative as Anticipation	9
1.5.2	PID on the Surge Plant	11
1.5.3	The Noise Problem	11
1.6	Cascade Control for Heading	11
1.6.1	The Problem with a Single Heading Loop	11
1.6.2	The Cascade Architecture	13
1.6.3	Quantitative Analysis	14
1.7	Feedforward: Using What You Know	14
1.7.1	Motivation	14
1.7.2	Two Forms of Feedforward	15
1.7.3	Application to the USV	15
1.8	Practical Modifications Beyond Ideal PID	17
1.8.1	Anti-Windup	17
1.8.2	Derivative on Measurement	18
1.8.3	Derivative Filtering	19
1.8.4	Output Slew Rate Limiting	20
1.8.5	Setpoint Prefiltering and Trajectory Constraints	21
1.9	Tuning Methods	22
1.9.1	Two Categories of Parameters	22

1.9.2	Linear Transfer Function Model	23
1.9.3	The Case for Intuition First	23
1.9.4	A Systematic Tuning Process	24
1.9.5	Troubleshooting by Symptom	24
1.10	Summary	25

Chapter 1

PID Control: From Analysis to Practice

1.1 Introduction

Every controlled system faces the same fundamental problem: you want an output to follow a desired value, but the plant doesn't do what you ask unless you continuously correct it. A ship asked to hold a heading will drift off course in wind and current. A vessel asked to cruise at a fixed speed will slow down as it climbs a river current. Open-loop control — computing a fixed input and hoping for the best — only works when your model is perfect and disturbances don't exist.

Feedback control addresses this by closing the loop: measure the output, compare it to the desired value, and use that *error* to drive the input. The question then becomes: exactly what do you do with the error? The answer that has dominated engineering practice for nearly a century is *proportional-integral-derivative (PID) control*.

PID is the workhorse of industrial and marine control systems. The autopilot keeping a surface ship on course, the speed governor on a submarine's propulsion system, the altitude-hold function on an aircraft's flight control computer — these all rely on variants of the same three-term structure. Its staying power comes from three practical virtues:

1. Simple to understand and implement
2. Can work without a detailed mathematical model of the plant — although a model can be a huge help in understanding.
3. The three parameters can be tuned in the field by engineers who understand what each one does.

That last point — understanding what each parameter *does* — is what this article is about. Most treatments of PID present the equations and then move quickly to formulas for choosing gains. This article takes the opposite approach: we will spend most of our time building physical intuition for each term before touching a tuning method. The goal is that when you

look at a poorly performing response, you can diagnose what is wrong and know which knob to turn.

Running Examples

Throughout this article we will use two running examples drawn directly from the USV lab.



Figure 1.1: The USV used in the lab.

Surge (speed) control. The USV’s forward speed response to throttle can be approximated by a first-order transfer function. The objective is to hold a desired speed in the presence of disturbances like current and wind — it is a *constant-setpoint tracking problem*, the simplest and most common control scenario.

Heading control. The USV’s yaw-rate response to rudder is also first-order, but the performance metric we care about is *heading* (yaw angle), which is the integral of yaw rate. This mismatch between the plant output and the performance metric turns out to be the key motivation for *cascade control*: an inner loop that regulates yaw rate quickly, wrapped inside an outer loop that uses that fast inner response to achieve precise heading. This is also the architecture used by ArduRover.

1.2 The PID Controller

1.2.1 Intuition Before Math

The PID controller computes a corrective input from three components of the error $e(t) = r(t) - y(t)$, where $r(t)$ is the reference (also called the *command* or *setpoint*) and $y(t)$ is the measured output.

Proportional (K_P) React to the current error. If the vessel is 2 m/s below target speed, apply a corrective throttle proportional to that gap. Larger error, larger correction.

Integral (K_I) Adapt to the history of error. If a river current has been persistently slowing the vessel, the integrator builds up a sustained correction over time to compensate.

Derivative (K_D) Anticipate the trend. If the speed error is shrinking rapidly — the vessel is closing in on its target — the derivative term begins reducing the throttle command before the speed overshoots.

A useful analogy is a driver trying to hold a fixed highway speed. The P term is how firmly they press the pedal in proportion to how far off-speed they are. The I term represents what the driver has learned about the road grade and headwind — accumulated experience that shows up as a steady baseline pressure on the pedal. The D term is the instinctive easing-off as the speedometer needle swings rapidly toward the target.

1.2.2 Transfer Function Form

The time-domain PID law is:

$$u(t) = K_P e(t) + K_I \int_0^t e(\tau) d\tau + K_D \dot{e}(t) \quad (1.1)$$

Taking the Laplace transform gives the controller transfer function:

$$C(s) = K_P + \frac{K_I}{s} + K_D s = \frac{K_D s^2 + K_P s + K_I}{s} \quad (1.2)$$

Two features are worth noting. First, there is one pole at $s = 0$: the integrator. We will see in Section 1.4 exactly why this eliminates steady-state error. Second, there are two zeros, at the roots of $K_D s^2 + K_P s + K_I = 0$. The designer places these anywhere in the complex plane by choosing K_P , K_I , and K_D — significant freedom to shape the closed-loop response.

Note that $K_D s$ grows without bound as $s \rightarrow \infty$. This means the ideal PID transfer function is *improper* and cannot be implemented exactly in hardware or software. In practice, the D term always includes a high-frequency limit; we will address this in Section 1.8.

1.2.3 Closed-Loop Transfer Function

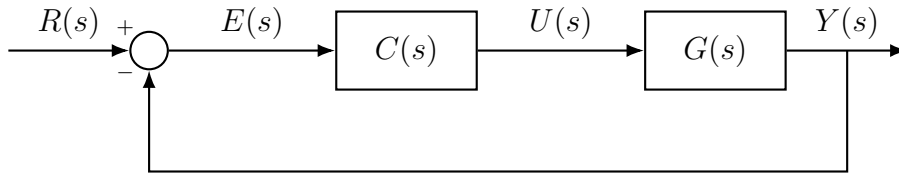


Figure 1.2: Standard unity-feedback control loop.

For the loop in Fig. 1.2, the output is $Y = C(s)G(s)E$ and the error is $E = R - Y$. Substituting:

$$Y = C(s)G(s)(R - Y) \implies Y(1 + C(s)G(s)) = C(s)G(s)R$$

so the closed-loop transfer function is:

$$T(s) = \frac{Y(s)}{R(s)} = \frac{C(s)G(s)}{1 + C(s)G(s)} \quad (1.3)$$

The order of the closed-loop equals the number of poles in $C(s)G(s)$. A PID controller adds one pole (the integrator at $s = 0$) to whatever poles the plant already has. For our first-order surge plant, the closed-loop will be second-order once integral action is included.

The zeros of $C(s)$ also appear as zeros of $T(s)$. Controller zeros speed up the transient response and can introduce overshoot — something to keep in mind as we add integral and derivative terms.

1.3 Proportional Control on a First-Order Plant

1.3.1 The USV Surge Model

Linear analysis of the surge dynamics (forward speed vs. throttle) yields a first-order transfer function:

$$G(s) = \frac{K}{\tau s + 1} \quad (1.4)$$

where K is the steady-state speed per unit throttle (m/s per unit effort) and τ is the time constant set by the balance of inertia and drag. Throttle is normalized to ± 1 , with $+1$ being full forward effort and -1 full reverse.

1.3.2 Closed-Loop with P Only

With a pure proportional controller $C(s) = K_P$, the open-loop transfer function is $C(s)G(s) = K_P K / (\tau s + 1)$. Substituting into Eq. (1.3):

$$T_P(s) = \frac{K_P K / (\tau s + 1)}{1 + K_P K / (\tau s + 1)} = \frac{K_P K}{\tau s + 1 + K_P K} = \frac{K_\infty}{\tau_{cl} s + 1} \quad (1.5)$$

where:

$$K_\infty = \frac{K_P K}{1 + K_P K}, \quad \tau_{cl} = \frac{\tau}{1 + K_P K} \quad (1.6)$$

The closed-loop is still first-order — proportional control does not change the system order. But compared to the open-loop plant, two things have changed: the time constant is smaller (faster response) and the DC gain is less than one.

The DC gain of the closed-loop system, K_∞ is less than one, which is the root of the problem. For a unit step in desired speed, the steady-state output is K_∞ , so the *steady-state error* is:

$$e_{ss} = 1 - K_\infty = \frac{1}{1 + K_P K} \quad (1.7)$$

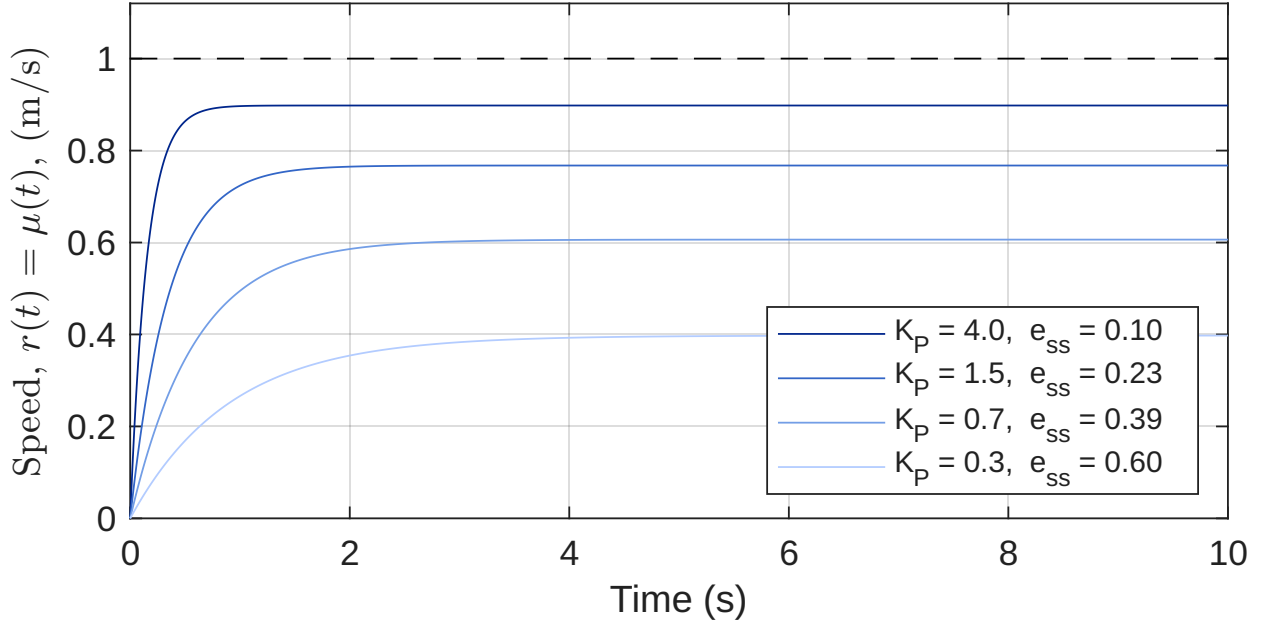


Figure 1.3: Step response of the P-controlled surge plant for increasing proportional gain. Higher gain improves speed and reduces error, but steady-state error persists regardless.

Figure 1.3 illustrates the trade-off: increasing K_P makes the response faster and reduces steady-state error, but the error never reaches zero. To eliminate it entirely, we need a different kind of controller action.

One more note: a first-order plant with proportional-only control cannot become unstable. The closed-loop pole is at $s = -(1 + K_P K)/\tau$, which is always in the left half-plane for positive gains. The only cost of increasing K_P is noisier, jerkier actuator behavior — not instability. This will not be true once the plant order increases or a second loop is closed.

1.4 Adding Integral Action

1.4.1 Why Integral Eliminates Steady-State Error

The intuitive argument is straightforward. At steady state, all signals are constant and $\dot{u} = 0$. The integral term contributes $K_I \int e dt$ to the control output. For this contribution to be constant, its integrand must be zero — meaning $e = 0$. Any nonzero steady-state error would keep the integrator growing, contradicting the assumption of steady state. Therefore, if the loop is stable, the steady-state error to a step reference must be zero.

More formally: the PI controller has a pole at $s = 0$, making the open-loop transfer function Type 1. By the final value theorem, a Type 1 loop tracks step references with zero steady-state error.

1.4.2 PI on the Surge Plant

With $C(s) = K_P + K_I/s = (K_P s + K_I)/s$, the open-loop transfer function is:

$$C(s)G(s) = \frac{K(K_P s + K_I)}{s(\tau s + 1)}$$

The closed-loop transfer function is:

$$T_{PI}(s) = \frac{K(K_P s + K_I)}{\tau s^2 + (1 + K K_P)s + K K_I} \quad (1.8)$$

The denominator — the *characteristic polynomial* — is now second-order. Writing it in standard form $s^2 + 2\zeta\omega_n s + \omega_n^2$ (after dividing by τ):

$$\omega_n = \sqrt{\frac{K K_I}{\tau}}, \quad \zeta = \frac{1 + K K_P}{2\sqrt{\tau K K_I}} \quad (1.9)$$

These two expressions directly guide design:

- K_I controls the natural frequency ω_n . Choose K_I to set the desired speed of response: $K_I = \omega_n^2 \tau / K$.
- K_P primarily controls the damping ratio ζ (through the $(1 + K K_P)$ numerator term). Once K_I is fixed, choose K_P to target a desired ζ : $K_P = (2\zeta\omega_n \tau - 1)/K$.
- For $\zeta < 1/\sqrt{2} \approx 0.707$, the response will overshoot. For $\zeta > 1$, it will be overdamped. A common target is $\zeta \approx 0.707$, which gives near-critical damping with no significant overshoot.

The closed-loop also has a zero at $s = -K_I/K_P$, inherited from the PI controller. This zero does not affect the stability (poles determine that), but it does pull the step response slightly faster and may add a small amount of overshoot beyond what the poles alone would predict. The effect is larger when the zero is close to the imaginary axis — that is, when K_I/K_P is small. This reinforces a useful design heuristic: add integral action only when needed to eliminate steady-state error, and keep K_I small relative to K_P . A large K_I pulls the zero close to the origin, amplifies the overshoot contribution of the zero, and risks the windup problem described below.

1.4.3 Integral Windup

The integrator is powerful, but it has a failure mode. When the actuator saturates — throttle clipped at its maximum value — the plant input is no longer following the controller output. The error persists, the integrator keeps growing, and by the time the plant output finally approaches the setpoint, the integrator has accumulated a large value that drives a large overshoot before it can unwind. This is *integral windup*.

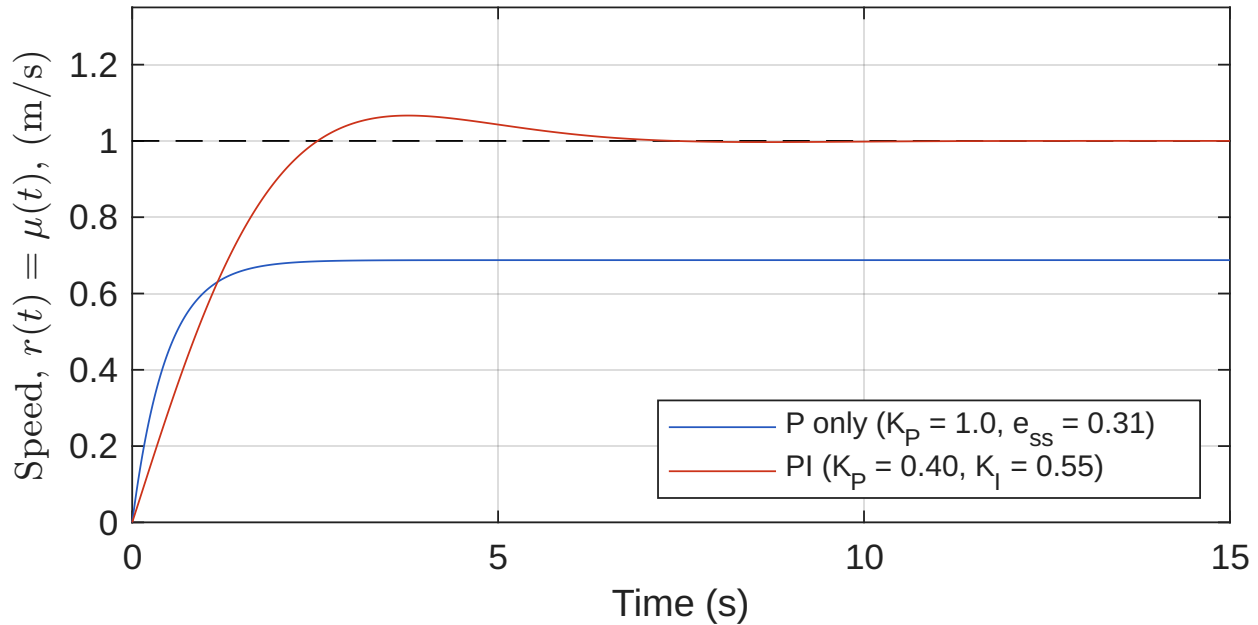


Figure 1.4: Comparison of P-only and PI step responses on the surge plant.

A concrete scenario: the USV is commanded to accelerate from rest to full cruise speed. The throttle saturates at its maximum for the entire ramp-up. During this time the error is large and positive, and the integrator accumulates a large value. When the speed finally reaches the setpoint, the controller output is still large because of the accumulated integral, pushing the throttle above what is needed and causing the vessel to overshoot before the integrator can recover. Section 1.8 addresses fixes.

1.5 Derivative Action

1.5.1 Derivative as Anticipation

The derivative term $K_D \dot{e}(t)$ responds to the *rate of change* of error, not its current value. When the error is decreasing rapidly — the output is converging on the setpoint quickly — the derivative term provides a braking action that reduces overshoot. The analogy is a shock absorber: it resists velocity, not displacement.

Unlike the proportional term, which reacts to where you are, and the integral, which reacts to where you have been, the derivative reacts to where you are going.

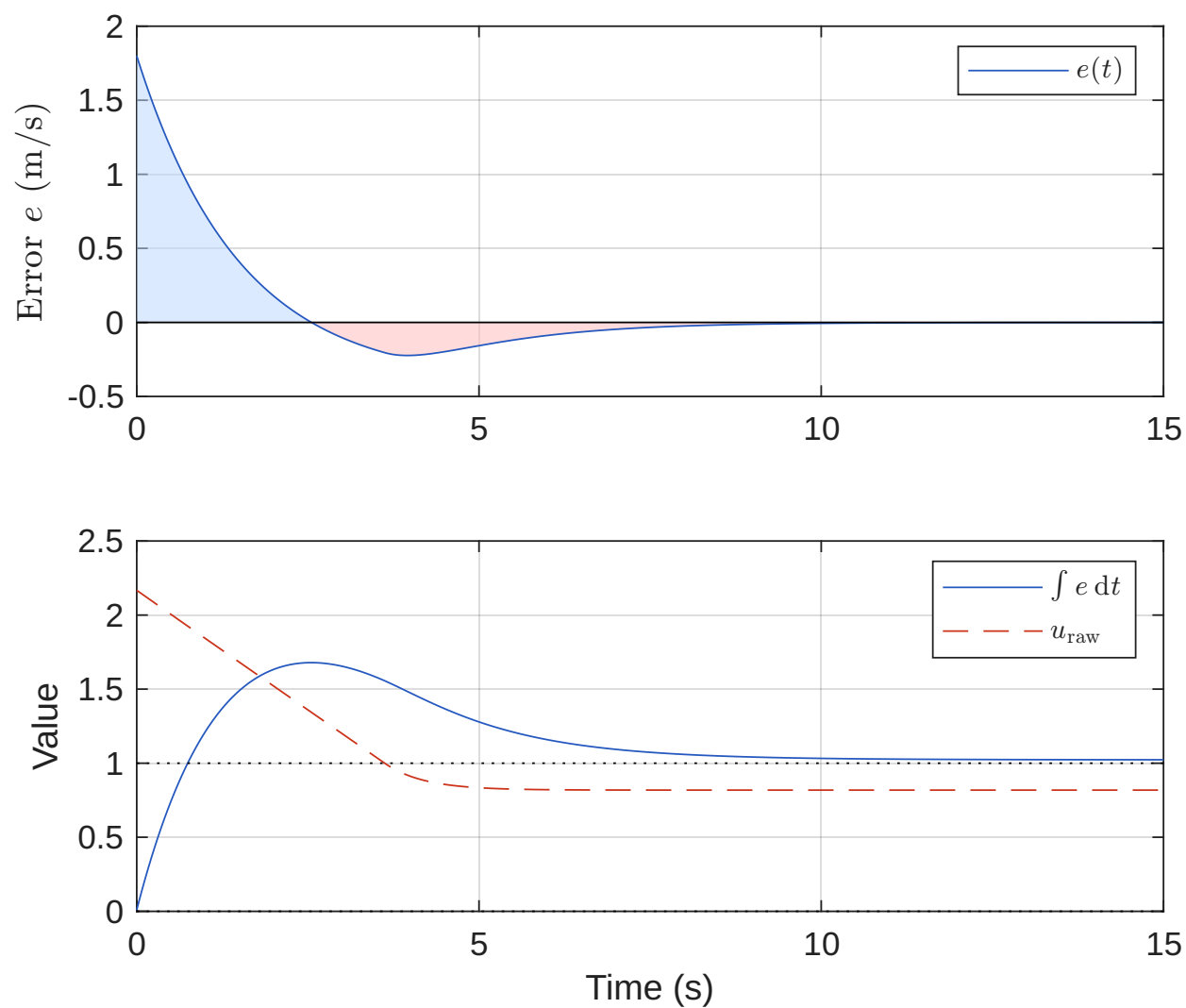


Figure 1.5: Integral windup: the area accumulated under $e(t)$ during saturation must be “paid back,” causing overshoot and slow recovery once the output leaves saturation.

1.5.2 PID on the Surge Plant

Adding the derivative term to PI gives the full PID controller. The characteristic polynomial of the closed-loop becomes:

$$\begin{aligned}\Delta(s) &= s(\tau s + 1) + K(K_D s^2 + K_P s + K_I) \\ &= (\tau + K K_D) s^2 + (1 + K K_P) s + K K_I\end{aligned}\tag{1.10}$$

Comparing to the PI case (1.8), the derivative term increases the coefficient of s^2 from τ to $\tau + K K_D$. This has two effects on the natural frequency and damping ratio:

$$\omega_n = \sqrt{\frac{K K_I}{\tau + K K_D}}, \quad \zeta = \frac{1 + K K_P}{2\sqrt{(\tau + K K_D) K K_I}}\tag{1.11}$$

The D term *lowers* ω_n and *raises* ζ . In physical terms: derivative action slows the natural frequency slightly while adding damping. If a PI-controlled system is too oscillatory (low ζ), adding K_D is one way to fix it.

On a first-order plant, however, K_P already provides a path to increasing ζ (via the $1 + K K_P$ numerator term in ζ). The derivative term provides an additional handle but is not strictly necessary. For second-order plants, where the P term alone cannot independently set both ω_n and ζ , derivative action is far more valuable. This is the general rule: D matters most when the plant itself has dynamics that limit the damping achievable with P and I alone.

1.5.3 The Noise Problem

The ideal D term $K_D s$ has a transfer function whose gain grows without bound at high frequencies. Sensor noise consists of rapid, small fluctuations. An unfiltered D term amplifies these into large, noisy actuator commands — a propeller being driven in rapid small oscillations around its mean command. This issue, the *dirty derivative*, is why pure derivative is never implemented in practice, and why the D term is often omitted entirely in noisy systems.

The solution is discussed in Section 1.8: add a first-order low-pass filter to the D term that caps its gain at high frequencies while preserving its damping effect at the frequencies of interest.

1.6 Cascade Control for Heading

1.6.1 The Problem with a Single Heading Loop

The USV's yaw dynamics are well-modeled by a first-order relationship between rudder input and yaw rate $\dot{\psi}$:

$$G_r(s) = \frac{K_r}{\tau_r s + 1}\tag{1.12}$$

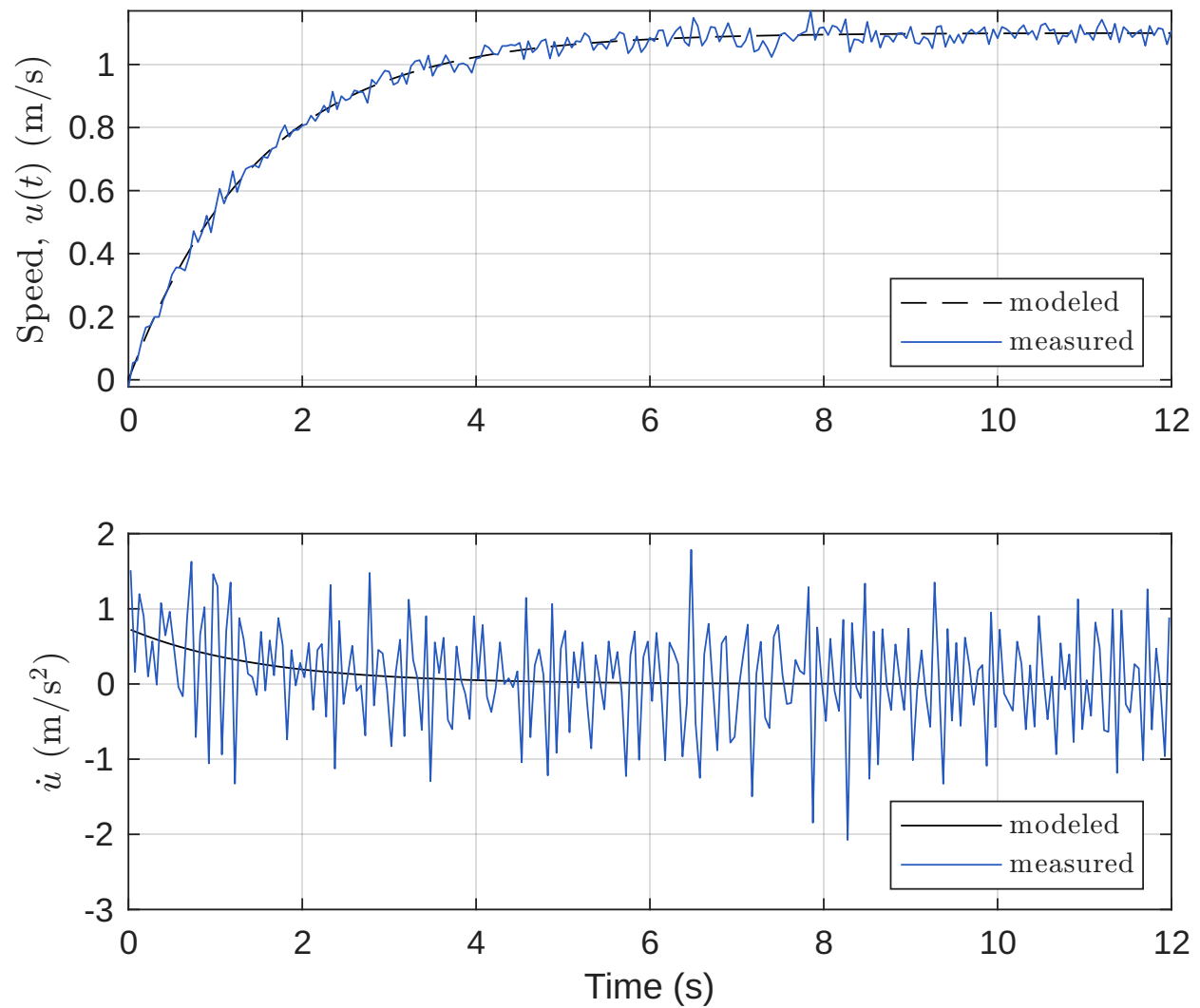


Figure 1.6: Sensor noise amplification by the derivative term. Small fluctuations in the measurement are amplified, becoming large spikes in $\dot{u}$.

The performance objective, however, is heading ψ — the integral of yaw rate. Heading is related to rudder by:

$$G_\psi(s) = \frac{G_r(s)}{s} = \frac{K_r}{s(\tau_r s + 1)} \quad (1.13)$$

This is a second-order, Type 1 plant (one integrator from the kinematics). Applying a simple proportional heading controller $C_\psi(s) = K_\psi$:

$$C_\psi G_\psi = \frac{K_\psi K_r}{s(\tau_r s + 1)}$$

$$\omega_n = \sqrt{\frac{K_\psi K_r}{\tau_r}}, \quad \zeta = \frac{1}{2\sqrt{K_\psi K_r \tau_r}} \quad (1.14)$$

Notice that $\zeta \propto 1/\sqrt{K_\psi}$. Increasing the gain to get a faster heading response simultaneously reduces damping. With only one parameter (K_ψ), the designer cannot independently choose both speed and damping — a fundamental limitation of the single-loop approach.

1.6.2 The Cascade Architecture

Cascade (inner/outer loop) control resolves this by splitting the heading problem into two simpler problems:

1. **Inner loop** (yaw rate): a fast controller $C_r(s)$ tracks a commanded yaw rate $\dot{\psi}_{\text{des}}$. The plant is the first-order $G_r(s)$ — straightforward to control well with PI.
2. **Outer loop** (heading): a slow controller $C_\psi(s)$ computes the yaw-rate command based on the heading error. It “sees” the inner loop as its plant.

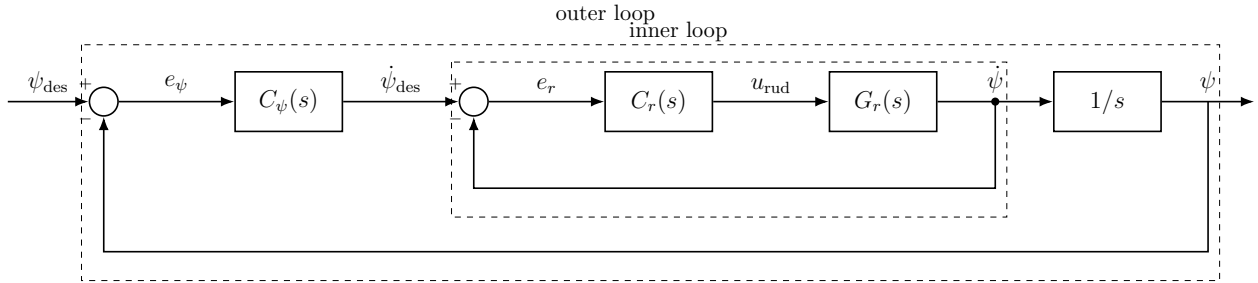


Figure 1.7: Cascade control structure for heading. The inner loop regulates yaw rate; the outer loop regulates heading by commanding a desired yaw rate.

The fundamental design rule is: **the inner loop must respond significantly faster than the outer loop.** In terms of step response, this means the inner loop’s closed-loop time constant τ_{cl} should be at least 5 times smaller than the outer loop’s desired response time. Put differently: if the vehicle should reach a commanded heading in roughly T seconds, the yaw-rate loop should reach a commanded yaw rate in roughly $T/5$ seconds or less. If the inner loop is slow, it distorts the signals the outer loop is trying to control and the approximation in the next section breaks down.

1.6.3 Quantitative Analysis

Suppose the inner yaw-rate loop is well-tuned and its closed-loop transfer function $T_{\text{inner}}(s)$, approximates a first-order response:

$$T_{\text{inner}}(s) = \frac{\dot{\Psi}(s)}{\dot{\Psi}_{\text{cmd}}(s)} \approx \frac{1}{\tau_{\text{cl}} s + 1} \quad (1.15)$$

where τ_{cl} is small (inner loop is fast). The outer loop then sees an effective plant of:

$$G_{\text{eff}}(s) = \frac{T_r(s)}{s} \approx \frac{1}{s(\tau_{\text{cl}} s + 1)} \quad (1.16)$$

As $\tau_{\text{cl}} \rightarrow 0$, this approaches $1/s$ — a pure integrator. With a proportional outer controller $C_\psi(s) = K_\psi$, the outer closed-loop heading response becomes:

$$T_\psi(s) \approx \frac{K_\psi/s}{1 + K_\psi/s} = \frac{K_\psi}{s + K_\psi} \quad (1.17)$$

This is approximation (which assumes τ_{cl} is vanishingly small) is *first-order*, so the closed-loop response is a clean monotone heading response with time constant $1/K_\psi$ and no overshoot. The DC gain of Eq. (1.17) is unity ($T_\psi(0) = 1$), confirming zero steady-state error for a step heading command — the kinematic integrator in the plant makes the outer loop type-1 even with a proportional-only outer controller. The single-loop problem of coupled speed and damping has disappeared: the outer loop simply has one parameter (K_ψ) that sets the heading response speed, and the inner loop's good damping ensures the approximation holds.

If we remove our assumption of a vanishingly small internal response ($\tau_{\text{cl}} \rightarrow 0$) and include the finite inner-loop time constant τ_{cl} , the outer characteristic polynomial is now the quadratic $\tau_{\text{cl}} s^2 + s + K_\psi$, giving:

$$\omega_n = \sqrt{K_\psi/\tau_{\text{cl}}}, \quad \zeta = \frac{1}{2\sqrt{K_\psi \tau_{\text{cl}}}}$$

The coupling between ω_n and ζ reappears, but with τ_{cl} in place of τ_{inner} . Since the inner loop is much faster than the yaw plant ($\tau_{\text{cl}} \ll \tau_r$), the outer loop natural frequency is much higher for the same outer gain, and the approximation $T_r \approx 1$ is valid across the outer loop's operating range.

In ArduRover, the `ATC_STR_RAT_*` gains configure the inner yaw-rate loop; a separate navigation controller acts as the outer heading loop.

1.7 Feedforward: Using What You Know

1.7.1 Motivation

Feedback is inherently reactive; it waits for error to develop before generating a corrective signal. For a well-designed loop, this delay is small — but it is never zero. If you already

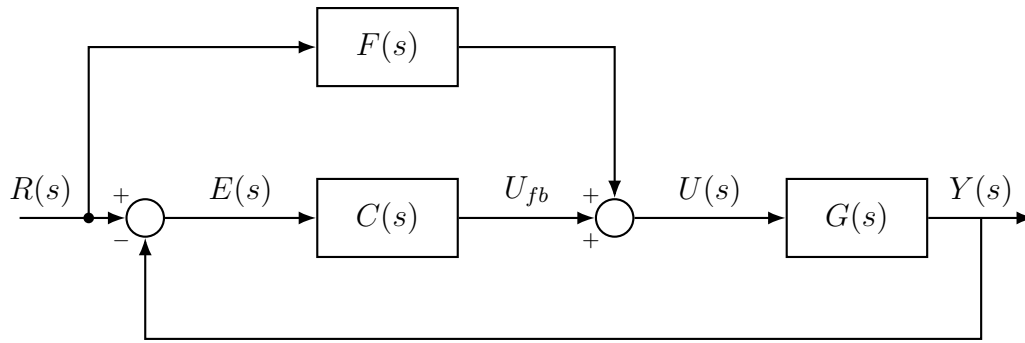


Figure 1.8: Feedforward augments feedback: the FF path provides a predictive baseline while feedback corrects residual error.

know (or can predict) the input required to achieve a desired output, you can supply that input directly, without waiting for error to accumulate. This is *feedforward*.

Feedforward and feedback complement each other. Feedforward handles the predictable portion of the control effort — the baseline required to achieve the desired output in a nominal, disturbance-free environment. Feedback handles what feedforward cannot: model error, disturbances, and uncertainty.

1.7.2 Two Forms of Feedforward

Operating-point bias is appropriate when the setpoint is approximately constant. Rather than expressing feedforward as a gain, it is parameterized directly in physical terms: “at this desired speed, I need this much throttle.” The integral term then corrects whatever residual the bias gets wrong.

Proportional (command) feedforward scales with the reference signal. For a first-order plant with DC gain K , the steady-state input required to produce output r is simply $u_{\text{ff}} = r/K$. Applying this directly gives immediate response proportional to the command, without waiting for integral action to build. This is most useful when the setpoint varies continuously.

Both forms raise the same question: why not just use a large integrator instead? A large K_I can also track varying references and reject disturbances. The problem is that integral windup and aggressive closed-loop poles make large K_I difficult to tune without oscillation or overshoot. Feedforward achieves the same steady-state result with a much less aggressive feedback loop.

1.7.3 Application to the USV

Surge (speed) control. The ArduRover speed controller uses an operating-point bias, configured via two parameters: `CRUISE_SPEED` (the nominal desired speed, m/s) and `CRUISE_THROTTLE` (the throttle percentage needed to hold that speed). Together these establish the expected throttle at the operating point, accounting for the vehicle’s drag and propulsion efficiency.

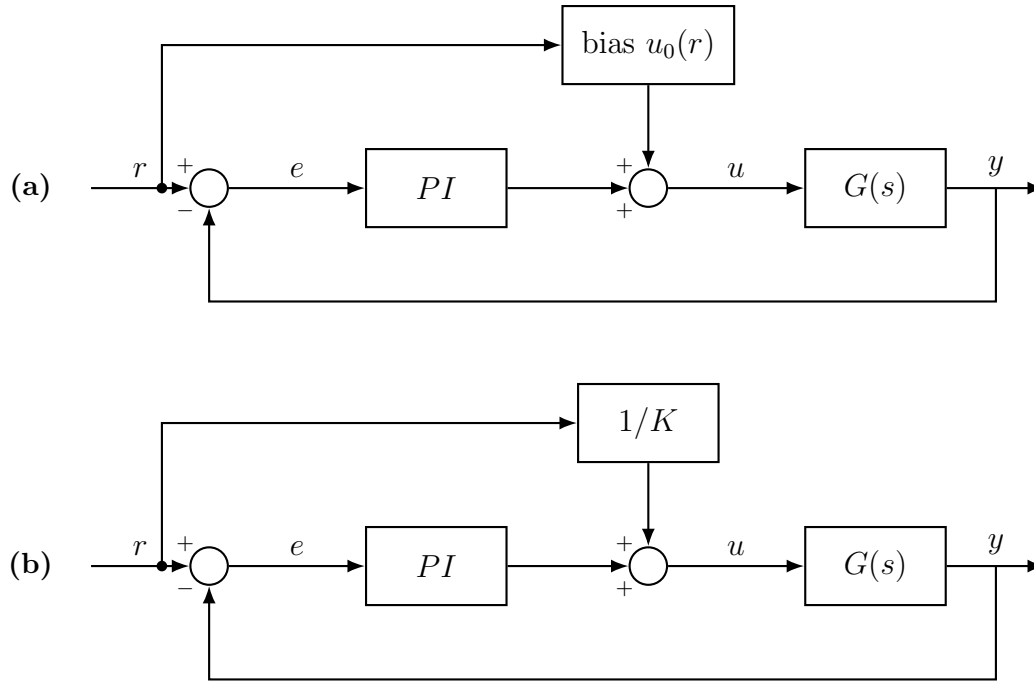


Figure 1.9: The two feedforward forms have different signal paths. Operating-point bias (a) contributes a fixed offset based on the setpoint value. Proportional feedforward (b) scales continuously with the reference.

The PI feedback loop then handles deviations from that point caused by currents, wind, and vehicle loading.

This approach works well because speed control is typically a constant-setpoint problem. When the desired speed changes significantly, the integral catches up; the operating-point bias only needs to be approximately right.

Heading (yaw rate) control. The steering controller uses proportional feedforward via `ATC_STR_RAT_FF`. During a turn, the commanded yaw rate varies continuously, and the FF term provides the primary control authority: rudder command $\approx FF \times \dot{\psi}_{\text{des}}$. The P and I gains are kept small relative to FF, handling only the residual between the commanded and actual yaw rate. The ideal starting value is `ATC_STR_RAT_FF` = $1/K_r$, the inverse of the yaw-rate plant DC gain from Eq. (1.12); field tuning refines it from that starting point to account for nonlinearities such as speed-dependent drag and rudder deadband.

This is the correct choice because yaw-rate commands vary widely during maneuvers — an operating-point bias (fixed offset) would provide no systematic benefit. The proportional FF directly inverts the plant’s DC gain and responds instantaneously to every change in the commanded rate.

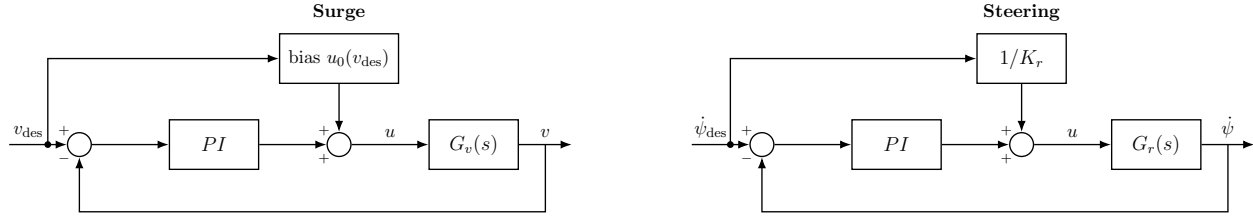


Figure 1.10: Feedforward configurations for surge (operating-point bias) and yaw-rate (proportional FF) control.

1.8 Practical Modifications Beyond Ideal PID

The sections above developed the ideal PID controller. Real implementations modify this ideal in several important ways. Because the lab asks you to both simulate controllers in MATLAB and configure them on the vehicle, each modification is discussed from two angles:

- **Linear analysis (MATLAB):** how to represent the modification as a transfer function so it can be incorporated into a closed-loop model and analyzed with the tools from this course.
- **ArduRover implementation:** how the same idea is realized in the autopilot, which typically uses a discrete, nonlinear version.

The two approaches produce qualitatively similar behavior; the linear model is an approximation that is useful for design and analysis, while the ArduRover implementation handles the practical realities of a digital controller with actuator limits.

1.8.1 Anti-Windup

The problem. Every actuator has limits. When the control output saturates (throttle pinned at maximum, rudder at its mechanical stop), the plant input is clipped to the saturation bound. The integrator, however, does not know this — it keeps integrating the error as if the full output were reaching the plant. By the time the output leaves saturation, the integrator has accumulated a value far beyond what is needed, causing a large overshoot and slow recovery. This is *integral windup*.

The fix: integrator clamping (ArduRover). The simplest anti-windup strategy stops integration whenever the output is saturated and the error would push the integrator further in the saturating direction. Mathematically: if $u > u_{\max}$ and $e > 0$, freeze the integral; if $u < u_{\min}$ and $e < 0$, freeze the integral. Otherwise integrate normally.

In ArduRover, integral limits are enforced through output clamping in the PID library; the `MOT_SLEWRATE` parameter (discussed below) provides complementary protection by preventing the output from changing so rapidly that the plant saturates in the first place.

The fix: back-calculation (linear analysis). Back-calculation is the anti-windup approach that lends itself to linear modeling. Rather than switching the integrator on and off (nonlinear), it adds a continuous feedback path from the difference between the saturated and unsaturated

outputs back to the integrator input:

$$\dot{x}_I = e(t) + \frac{1}{T_t}(u_{\text{sat}} - u_{\text{raw}}) \quad (1.18)$$

where T_t is the *tracking time constant* that sets how quickly the integrator unwinds. When the output is not saturated, $u_{\text{sat}} = u_{\text{raw}}$ and the extra term vanishes — the controller behaves as normal. When saturated, the feedback drives $\dot{x}_I$ toward zero, preventing further wind-up. A good starting point is $T_t = \sqrt{T_I T_D}$ (or simply $T_t = T_I$ for a PI controller).

The value of back-calculation for simulation is that the additional path can be drawn in a Simulink or MATLAB block diagram and analyzed as part of the linear loop structure. It does not require a conditional logic block.

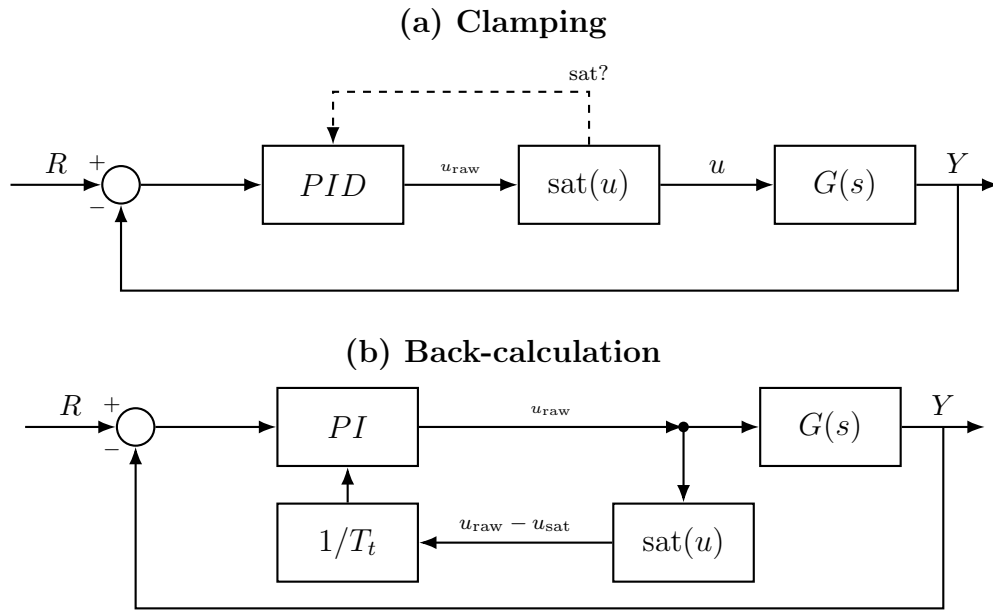


Figure 1.11: Two anti-windup methods. Clamping (a) is simpler to implement in code; back-calculation (b) can be represented as a linear feedback path for analysis.

1.8.2 Derivative on Measurement

The problem. When the reference $r(t)$ changes as a step, the error $e(t) = r(t) - y(t)$ also steps instantaneously. The derivative of a step is an impulse — so the D term briefly commands a very large (theoretically infinite) actuator signal. This *derivative kick* is especially disruptive for actuators like servos and propellers that can be damaged or destabilized by impulsive commands.

The fix. Compute the D term from the negative derivative of the *measurement* rather than the error:

$$u_D = -K_D \dot{y}$$

Since the measurement changes smoothly (the plant cannot respond instantaneously), $\dot{y}$ is well-behaved even when the reference steps. The proportional and integral terms are still computed from $e = r - y$ as usual.

Example. Consider a speed controller with $K_D = 0.2$ and a setpoint step from 1 m/s to 2 m/s at $t = 1$ s. With D on error: the error jumps from 0 to 1 m/s, so u_D spikes to $+0.2/\Delta t$ for one timestep (very large for a small Δt). With D on measurement: at $t = 1^+$, the measured speed is still 1 m/s and $\dot{y} \approx 0$, so $u_D \approx 0$. The derivative term only activates as the speed actually begins to change — which is exactly when its damping action is needed.

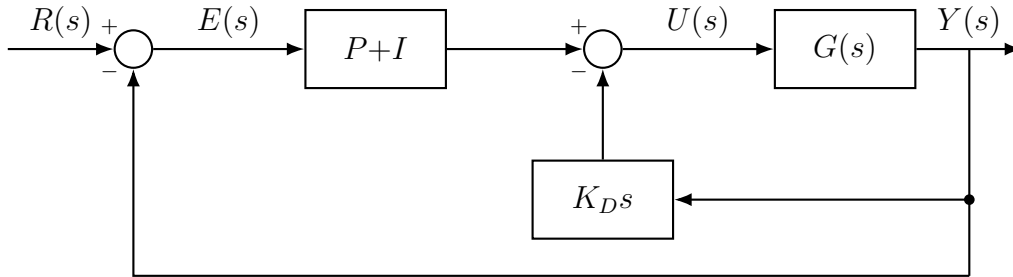


Figure 1.12: Derivative on measurement: the D term is computed from $-\dot{y}$ rather than $\dot{e}$, eliminating derivative kick on setpoint steps.

1.8.3 Derivative Filtering

The problem. Even with the derivative computed on the measurement, high-frequency noise in the sensor signal causes problems. The D term amplifies rapid changes, and sensor noise consists of exactly those rapid, small fluctuations. The result is large, fast actuator oscillations that wear out hardware and destabilize the plant.

The fix. Add a first-order lag in series with the differentiator:

$$D_f(s) = \frac{K_D s}{\tau_f s + 1} \quad (1.19)$$

To understand what this does, consider two extremes. For slowly varying signals (small s), the denominator $\tau_f s + 1 \approx 1$, and $D_f(s) \approx K_D s$ — pure differentiation, as intended. For rapidly varying signals (large s), the denominator dominates: $D_f(s) \approx K_D/\tau_f$, a constant gain. The D term no longer amplifies noise without bound; its gain is capped at K_D/τ_f for rapid changes.

The parameter τ_f is a tuning knob. A small τ_f passes high frequencies more readily — the D term is more responsive but also noisier. A large τ_f filters aggressively but blunts the D term's effectiveness at the frequencies of interest. In practice, τ_f is often set to $K_D/(5 \text{ to } 10)$ as a starting point.

Linear analysis (MATLAB). The filtered PID transfer function is:

$$C_f(s) = K_P + \frac{K_I}{s} + \frac{K_D s}{\tau_f s + 1} \quad (1.20)$$

This is a proper, rational transfer function that can be entered directly into MATLAB and used in any closed-loop analysis. When you cover frequency response later in the course,

Eq. (1.19) will look familiar as a *lead compensator* — a device that adds phase lead while bounding its high-frequency gain.

ArduRover implementation. ArduRover applies an exponential moving average (EMA) to the derivative measurement at each timestep, which is the discrete-time equivalent of the first-order lag in Eq. (1.19).

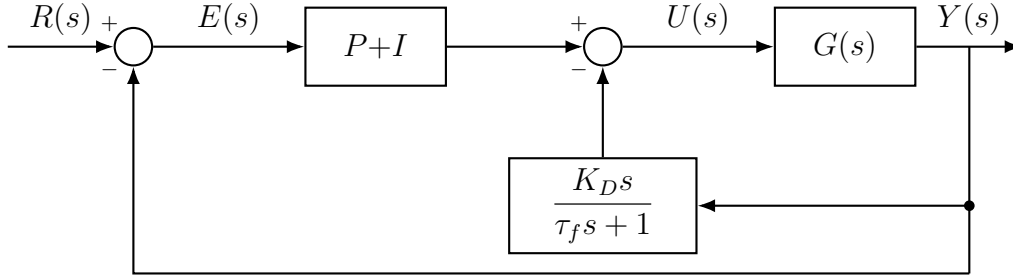


Figure 1.13: Filtered derivative: the first-order lag caps the D term gain at high frequencies, reducing noise amplification.

1.8.4 Output Slew Rate Limiting

The problem. Very rapid changes in controller output can excite dynamics that the PID model ignores: propeller cavitation, mechanical resonance in drive shafts, or current spikes that drain batteries and trigger overcurrent faults. A PID controller with high gains can, in principle, command the actuator to jump from one extreme to the other in a single timestep.

The fix (ArduRover). Limit the rate at which the controller output is allowed to change. If the desired output u_{raw} would require a change larger than $S_{\text{max}} \Delta t$ from the previous output, clamp it:

$$u = u_{\text{prev}} + \text{clamp}(u_{\text{raw}} - u_{\text{prev}}, -S_{\text{max}}\Delta t, +S_{\text{max}}\Delta t) \quad (1.21)$$

This does not reduce the ultimate steady-state value the output reaches; it only limits how quickly it gets there.

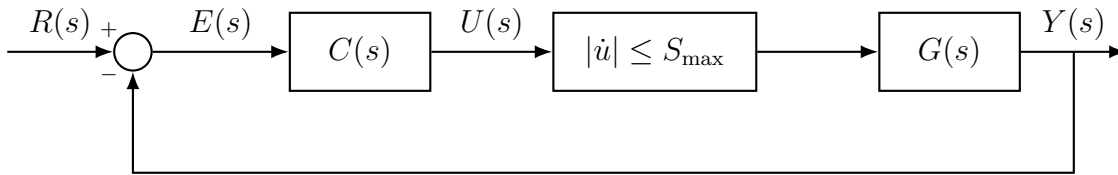


Figure 1.14: Slew rate limiter placed between the controller and the plant.

In ArduRover, `MOT_SLEWRATE` sets the maximum throttle change rate (units: fraction per second on a 0–100 scale). Setting it to zero disables slew limiting entirely.

Linear analysis (MATLAB). A slew rate limiter is nonlinear, but for small signals (where the rate limit is not active) it behaves like a unity gain. For linear analysis, a first-order lag

on the controller output is a tractable approximation:

$$U_{\text{plant}}(s) = \frac{1}{\tau_s s + 1} U_{\text{raw}}(s) \quad (1.22)$$

Choose τ_s to reflect the effective smoothing: if the slew rate is S_{max} (units/s) and the full actuator range is Δu , then $\tau_s \approx \Delta u / (2S_{\text{max}})$ is a reasonable estimate. This first-order lag adds a pole to the open-loop transfer function and can reduce the phase margin — an important check when tuning aggressive gains.

1.8.5 Setpoint Prefiltering and Trajectory Constraints

The problem. Step changes in the reference signal are a useful tool for system identification and testing — a sudden speed or heading command reveals the plant dynamics cleanly. In operation, however, commanding an instantaneous jump in speed asks the vehicle to produce infinite acceleration: physically impossible, and a guarantee that the actuator will saturate and the integrator will wind up before the plant can begin to respond.

More broadly, there is a mismatch between what the *controller* asks for (instantaneous change) and what the *plant* can deliver (bounded acceleration). Driving the controller hard against a limit it cannot satisfy activates dynamics that are harder to control — saturated actuators, windup, and sharp transients that can excite structural resonances.

The fix. The problem is not in the feedback loop but in the reference signal. Replace the step command with a smoothed reference that changes at a rate the vehicle can actually achieve. For speed control, a_{max} is literally an acceleration limit (m/s²): the commanded speed ramps up at no more than a_{max} meters per second per second. For heading, it is a turning rate limit. This is not a modification to the PID controller itself, but to the signal that feeds it.

Two standard implementations exist, and they produce qualitatively similar results with different shapes:

Linear analysis (MATLAB) — first-order reference filter. A first-order low-pass filter on the reference is the linear option:

$$R_f(s) = \frac{1}{\tau_r s + 1} R(s) \quad (1.23)$$

For a step in R , the filtered reference R_f rises exponentially with time constant τ_r rather than instantaneously. This can be added to the front of any MATLAB transfer function model, and the overall closed-loop from R to Y becomes $T(s)/(\tau_r s + 1)$. Choose τ_r to keep the demanded acceleration within the vehicle's capability.

ArduRover — rate limiter. A rate limiter converts a step command into a ramp: the reference increases linearly at a_{max} until it reaches the new setpoint, then holds. This is nonlinear (the output slope is capped regardless of how large the step is) but provides a hard, predictable bound on the demanded acceleration. The shapes differ: the low-pass filter gives an exponentially decelerating approach, while the rate limiter gives a constant-rate

ramp followed by a sharp corner. For analysis, the low-pass filter is the better model; for deployment, ArduRover’s rate limiter provides more predictable actuator loading.

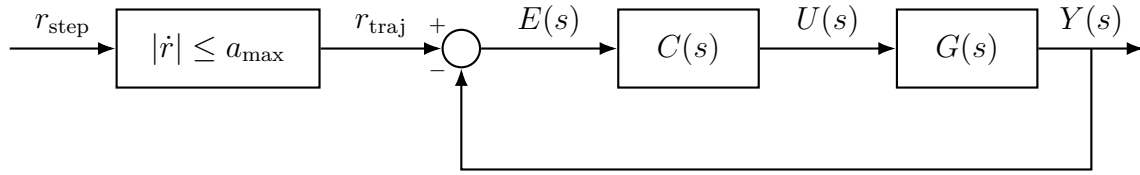


Figure 1.15: Setpoint prefilter: a rate limiter on the reference prevents the controller from being driven against actuator limits.

In ArduRover, `ATC_ACCEL_MAX` and `ATC_DECEL_MAX` set the maximum acceleration and deceleration rates for the speed command. Setting asymmetric values (e.g., slower deceleration limit for a vehicle with weak braking) is a natural extension of the rate-limiter concept.

1.9 Tuning Methods

1.9.1 Two Categories of Parameters

Before selecting any gains, it helps to recognise that the ArduRover speed controller has two fundamentally different kinds of tunable parameters.

Category 1 — Set from system knowledge. These parameters have correct values that can be determined analytically or read directly from hardware specifications, without iteration on the vehicle:

- *Input filter cutoffs* (`_FLT`, acceleration limit): matched to the desired reference bandwidth and vehicle performance limits.
- *Error and derivative filter cutoffs* (`_FLTE`, `_FLTD`): chosen from sensor noise characteristics (sample rate, noise floor).
- *Anti-windup limit* (`_IMAX`): set to the actuator saturation bound, which is a known hardware property.
- *Baseline feedforward* (`throttle baseline` or `_FF`): the linearised steady-state throttle required to hold cruise speed, derivable from the plant DC gain K_v . Use one or the other; they are redundant.
- *Derivative feedforward* (`_DFF`): initial value from the plant model; refined in field testing.

Category 2 — Gains tuned iteratively. These cannot be set analytically without a precise nonlinear model, and are best found by simulation first, then refined on the vehicle:

- Proportional, integral, and derivative gains (K_P , K_I , K_D — `_P`, `_I`, `_D`).

1.9.2 Linear Transfer Function Model

Simulation against the linear plant model allows Category 2 gains to be explored safely before going to the water. Figure 1.16 shows the full speed controller recast as a block diagram of transfer functions; all nonlinear elements are linearised as noted in Table 1.1.

Table 1.1: Linearisation of nonlinear ArduRover elements for simulation.

Element	Nonlinearity	Linear approximation
Acceleration limit	Rate limiter	First-order low-pass $\omega_a/(s + \omega_a)$
Throttle baseline	Lookup table	Gain K_{base} (slope at operating point)
Anti-windup $_IMAX$	Saturation	Omitted (assume unsaturated)
Dmod / PD scale	Slew limiter $\times$ gain	Combined gain K_{pd}

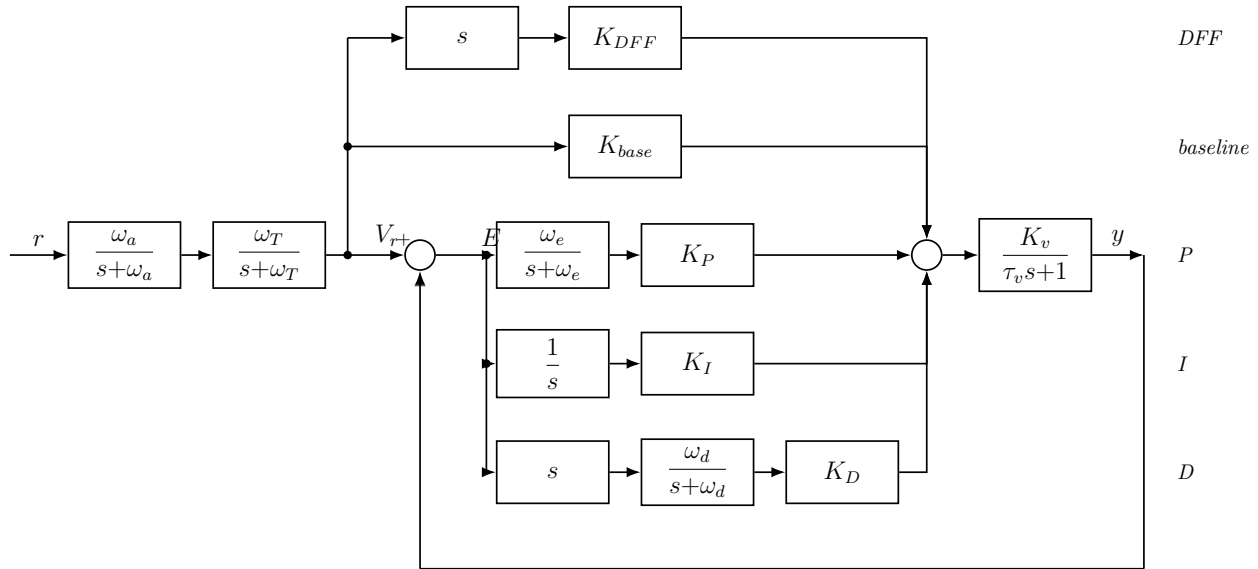


Figure 1.16: Linear transfer function model of the ArduRover speed controller. Input shaping: $\omega_a/(s+\omega_a)$ (acceleration limit) and $\omega_T/(s+\omega_T)$ ($_FLT$) produce the filtered reference V_r . Feedforward paths act on V_r directly; feedback paths act on the error $E = V_r - y$. The error filter $\omega_e/(s+\omega_e)$ ($_FLTE$) applies only to the P path; the D-term filter $\omega_d/(s+\omega_d)$ ($_FLTD$) attenuates derivative noise. Nonlinear elements are linearised per Table 1.1. The companion script `sim_speed_loop.m` implements this model in MATLAB for closed-loop simulation.

1.9.3 The Case for Intuition First

The sections above built the tools to answer a diagnostic question: given a poorly performing step response, which gain should change and in which direction? Flat steady-state error with a well-behaved transient: increase K_I or add integral. Fast oscillations that don't decay: reduce K_P or K_I . Slow convergence with no oscillation: increase K_P . Overshoot that takes many cycles to settle: reduce K_I or add K_D .

Table 1.2: Filter and input-shaping parameters of the linear speed-loop model. ArduRover parameter names are shown where applicable. Recommended values are starting points; the plant time constant τ_v comes from system identification.

Symbol	ArduRover param.	Description	Recommended start
τ_v	— (plant)	Surge time constant identified from sea trials	≈ 1.5 s (identified)
ω_a	ATC_ACCEL_MAX	First-order approximation of the trajectory acceleration limiter ($\omega_a = a_{\max}/v_{\text{cruise}}$)	1–2 rad/s
ω_T	_FLT_T	Low-pass filter on the target (reference) speed	$1/\tau_v - 2/\tau_v$ rad/s
ω_e	_FLTE	Low-pass filter on the speed error (P path only)	$3\text{--}5\omega_T$
ω_d	_FLTD	Low-pass filter on the derivative term (noise rejection)	$5\text{--}10\omega_T$

Good intuition makes systematic gain selection possible. The process below uses the identified plant model in MATLAB simulation, so mistakes are free.

1.9.4 A Systematic Tuning Process

1. **Start in simulation.** Build the closed-loop model in MATLAB using the identified transfer functions. All gain choices should be validated here before touching the vehicle.
2. **P only first.** Set $K_I = 0$ and $K_D = 0$. Increase K_P until the step response is fast with modest overshoot. Note the remaining steady-state error; the next step removes it.
3. **Add integral conservatively.** Use Eq. (1.9) with a modest target ω_n , then choose K_P for $\zeta \approx 0.7$. Verify the response has no more overshoot than the P-only case. Keep $K_I < K_P$.
4. **Add derivative only if needed.** For a first-order plant with PI, this is often unnecessary. If the PI response is too oscillatory and increasing K_P does not help, add a small K_D .
5. **Verify disturbance rejection.** Apply a step disturbance (simulated current or wind gust) and confirm recovery is acceptable.
6. **Transfer to the vehicle cautiously.** Start at roughly half the simulation gains, then increase in small steps while watching the actual response.

1.9.5 Troubleshooting by Symptom

Table 1.3 maps observable step-response symptoms to causes and corrective actions.

Table 1.3: PID troubleshooting guide.

Symptom	Likely cause	Action
Nonzero steady-state error, clean transient	$K_I = 0$ or too small	Add or increase K_I ; check anti-windup is not blocking integration
Slow rise, sluggish response	K_P too small	Increase K_P
Sustained oscillation	K_P or K_I too large	Reduce K_I first; then K_P if still oscillating
Overshoot with slow settling	K_I too large	Reduce K_I ; consider adding K_D for damping
Large overshoot on large steps	Integral windup during saturation	Enable anti-windup; add set-point prefilter
Noisy or chattering actuator	K_D too large or unfiltered	Reduce K_D ; increase τ_f in derivative filter
Actuator spike on set-point step	Derivative kick	Switch to derivative on measurement
Sluggish heading despite fast yaw rate	Inner loop too slow	Retune inner loop; reduce τ_{cl}

1.10 Summary

PID control is most useful when understood as a design progression, not a collection of independent tricks. Proportional control provides fast response but leaves steady-state error. Integral action eliminates that error by accumulating history, at the cost of potential windup and oscillation if K_I is too large. Derivative action adds damping — most valuable when the plant’s natural dynamics provide little of it.

Heading control on the USV illustrates that the choice of loop architecture matters as much as the choice of gains. A single feedback loop around the kinematic heading plant couples speed and damping in a way that limits performance; cascade control separates the problem into two well-posed first-order loops that can be tuned independently.

Feedforward shifts some of the burden from feedback to prediction. An operating-point bias handles the known, steady disturbance of drag at cruise; proportional feedforward handles continuously varying yaw-rate commands. In both cases, the feedback loop is left to handle what feedforward cannot: model error and disturbances.

The practical modifications in Section 1.8 are not edge cases — they are standard features of production controllers, and ArduRover implements each of them. Understanding why each modification exists is the difference between a controller that works in a lab and one that works on the water.

Table 1.4 maps the concepts developed in this article to their ArduRover counterparts.

Table 1.4: Concept-to-parameter mapping for ArduRover speed and steering control.

Concept	ArduRover Parameter	Section
Proportional gain (speed)	ATC_SPEED_P	Section 1.3
Proportional gain (steering)	ATC_STR_RAT_P	Section 1.3
Integral gain (speed)	ATC_SPEED_I	Section 1.4
Integral gain (steering)	ATC_STR_RAT_I	Section 1.4
Derivative gain (speed)	ATC_SPEED_D	Section 1.5
Derivative gain (steering)	ATC_STR_RAT_D	Section 1.5
Operating-point feedforward	CRUISE_THROTTLE, CRUISE_SPEED	Section 1.7
Proportional feedforward	ATC_STR_RAT_FF	Section 1.7
Anti-windup	Output clamping (internal)	Section 1.8
Output slew rate limit	MOT_SLEWRATE	Section 1.8
Trajectory constraint	ATC_ACCEL_MAX, ATC_DECEL_MAX	Section 1.8
Cascade inner loop	ATC_STR_RAT_*	Section 1.6
Cascade outer loop	Navigation controller	Section 1.6